



Batch: C3 Roll No.: 16010123325

**Experiment / Assignment / Tutorial
No. _2_**

Title: Implementation of linear Regression.

Aim: To understand & implement linear regression using python.

Expected Outcome of Experiment:

CO2: Describe and apply supervised learning methods.

Books/ Journals/ Websites referred:

1. <https://www.ibm.com/topics/linear-regression#:~:text=Resources-What%20is%20linear%20regression%3F,is%20called%20the%20independent%20variable.>
2. <https://www.geeksforgeeks.org/ml-linear-regression/>
3. <https://towardsdatascience.com/understand-the-data-with-univariate-and-multivariate-charts-and-plots-in-python-3b9fcd68cd8>

Theory of Linear Regression:

Linear regression analysis is used to predict the value of a variable based on the value of another variable. The variable you want to predict is called the dependent variable. The variable you are using to predict the other variable's value is called the independent variable. Linear Regression is a machine learning algorithm based on supervised learning. It performs a regression task. Regression models a target prediction value based on independent variables. It is mostly used for finding out the relationship between variables and forecasting. Different regression models differ based on – the kind of relationship between dependent and independent variables they are considering, and the number of independent variables getting used.



Problem Definition:

The objective of this experiment is to apply **Simple Linear Regression** and **Multiple Linear Regression** techniques on a given dataset in order to study the relationship between dependent and independent variables. The experiment aims to evaluate model performance using appropriate statistical metrics and to compare the predictive accuracy and explanatory power of simple and multiple regression models.

Apply Simple and Multiple Linear Regression on any dataset. Also display the appropriate metrics. Comment on you interpretation

Linear Regression:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,
mean_absolute_error, r2_score

#Import dataset
data = pd.read_csv("Iris.csv")
data.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
X = data[['PetalWidthCm']]
y = data[['PetalLengthCm']]

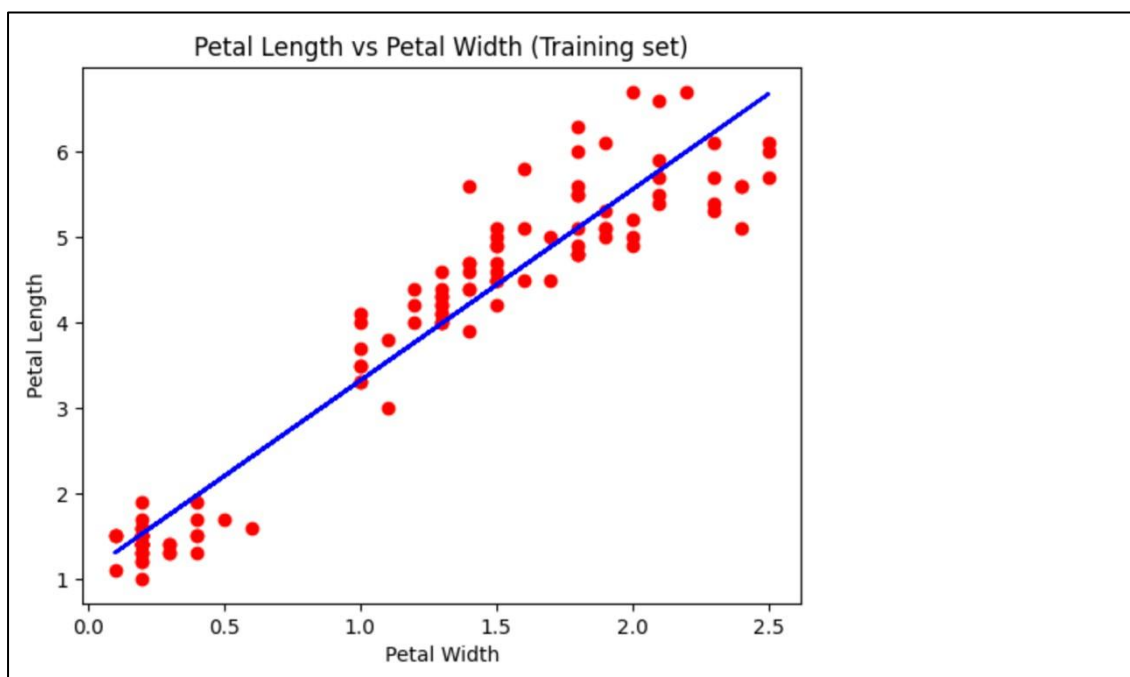
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
regressor = LinearRegression()  
regressor.fit(X_train, y_train)
```

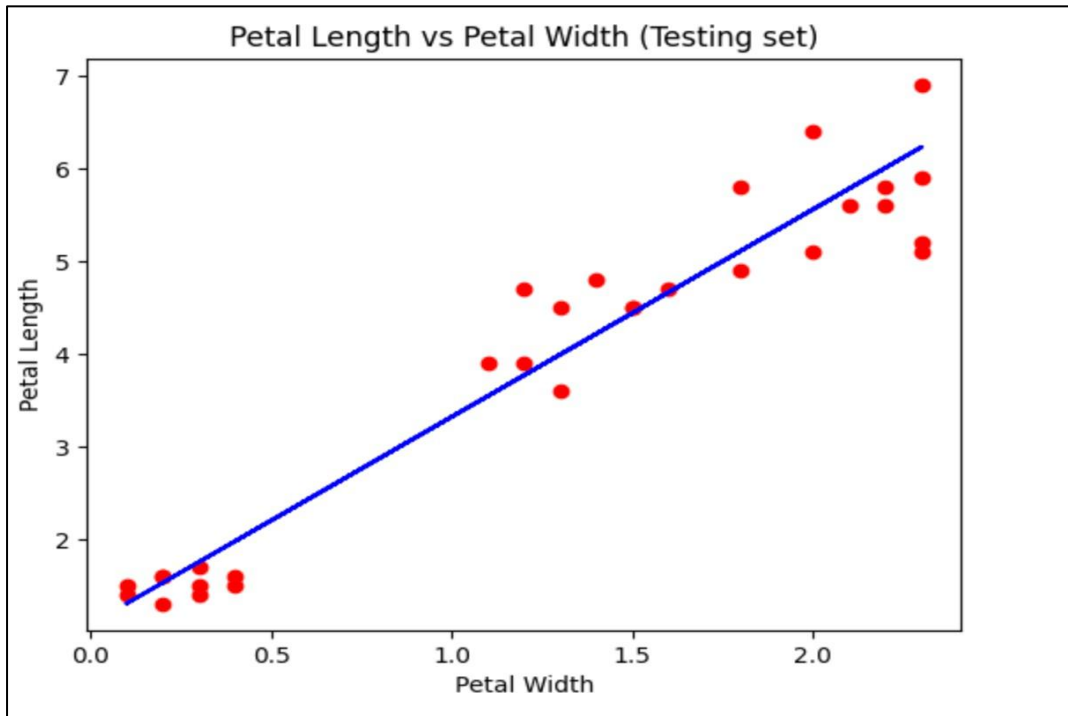
```
...  
▼ LinearRegression ⓘ ⓘ  
LinearRegression()
```

```
y_pred = regressor.predict(X_test)
```

```
#Petal Length vs Petal Width (Training set)  
plt.scatter(X_train, y_train, color='red')  
plt.plot(X_train, regressor.predict(X_train), color='blue')  
plt.title('Petal Length vs Petal Width (Training set)')  
plt.xlabel('Petal Width')  
plt.ylabel('Petal Length')  
plt.show()
```



```
plt.scatter(X_test, y_test, color='red')  
plt.plot(X_test, regressor.predict(X_test), color='blue')  
plt.title('Petal Length vs Petal Width (Testing set)')  
plt.xlabel('Petal Width')  
plt.ylabel('Petal Length')  
plt.show()
```



```
# Making a single prediction (for example the petal length for  
petal width=1.2 cm)  
print(regressor.predict([[1.2]]))  
np.set_printoptions(precision=2)
```

```
[3.76664779]  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:  
warnings.warn(
```

```
# Metrics  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
mae = mean_absolute_error(y_test, y_pred)  
r2 = r2_score(y_test, y_pred)  
  
print("Mean Squared Error=", mse)  
print("Mean Absolute Error=", rmse)  
print("R2 score=", r2)
```



Mean Squared Error= 0.23386845786901478
Mean Absolute Error= 0.48359948084030735
R2 score= 0.9286418347661203

Comments:

Relationship between variables

From the scatter plot of **Petal Width vs Petal Length**, a **strong positive linear relationship** is observed. As petal width increases, petal length also increases almost linearly.

Metrics Interpretation

- **Mean Squared Error (MSE):** Indicates the average squared difference between actual and predicted petal lengths. A **low MSE** shows that the model's predictions are close to the true values.
- **Root Mean Squared Error (RMSE):** RMSE is the square root of MSE and represents the error in the **same units as petal length**. A **small RMSE** indicates good prediction accuracy.
- **R² Score:** The R² value is **high (close to 1)**, which indicates that a large proportion of variance in petal length is explained by petal width. This confirms that petal width is a **strong predictor** of petal length.

Multiple Linear Regression:

```
import pandas as pd
import numpy as np

data = pd.read_csv("Iris.csv")
data.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
data['Species'] = le.fit_transform(data['Species'])

print(data['Species'])
```

```
... 0 0
    1 0
    2 0
    3 0
    4 0
    ..
145 2
146 2
147 2
148 2
149 2
Name: Species, Length: 150, dtype: int64
```

```
X = data[['SepalLengthCm', 'SepalWidthCm', 'PetalWidthCm',
'Species']]
y = data['PetalLengthCm']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.linear_model import LinearRegression
```

```
regressor = LinearRegression()  
regressor.fit(X_train, y_train)
```

```
... LinearRegression  
LinearRegression()
```

```
# Predict petal length for: Sepal length = 5.1, Sepal width = 3.5,  
Petal width = 0.2, Species = Setosa  
single_prediction = regressor.predict([[5.1, 3.5, 0.2, 0]])  
print("Predicted Petal Length:", single_prediction[0])
```

```
... Predicted Petal Length: 1.515049009226582  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739:  
warnings.warn(  
    _____
```

```
y_pred = regressor.predict(X_test)
```

```
coeff_df = pd.DataFrame(  
    regressor.coef_,  
    X.columns,  
    columns=['Coefficient']  
)  
print(coeff_df)
```

```
...      Coefficient  
SepalLengthCm    0.695072  
SepalWidthCm    -0.546683  
PetalWidthCm     1.079827  
Species          0.420193
```

```
least_contributing = coeff_df.abs().idxmin()  
print("Least contributing parameter:", least_contributing[0])
```

```
Least contributing parameter: Species
/tmp/ipython-input-1331032902.py:2: FutureWarning: Series.__getitem__
print("Least contributing parameter:", least_contributing[0])
```

```
from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)
n = X_test.shape[0]
p = X_test.shape[1]

adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

print("R-squared:", r2)
print("Adjusted R-squared:", adjusted_r2)
```

```
R-squared: 0.9634508585964104
Adjusted R-squared: 0.9576029959718361
```

Comments:

Interpretation of Regression Coefficients

- **Petal Width** shows a **strong positive coefficient**, indicating it is the **most influential predictor** of petal length.
- **Sepal Length** also contributes positively but to a lesser extent.
- **Sepal Width** has a comparatively **smaller coefficient**, suggesting weaker influence.
- **Species** has the **least contribution**, as most of the variation associated with species is already captured by petal measurements.

The parameter with the **smallest absolute coefficient** is identified as the **least contributing feature**.

Model Performance Metrics Interpretation

- **R-squared (R^2):**
The R^2 value is **high**, indicating that the independent variables collectively explain a large proportion of the variance in petal length.
- **Adjusted R-squared:**
The adjusted R^2 value is slightly lower than R^2 , accounting for the number of predictors used.
Since the adjusted R^2 remains high, it confirms that the additional variables improve the model without unnecessary complexity.



API/Function used:

Dataset Handling

- `pd.read_csv()` – to load the dataset
- `data.head()` – to display first few records

Data Preprocessing

- `LabelEncoder()` – to encode categorical variable (Species) (*Multiple Regression*)
- `fit_transform()` – to convert categorical data into numerical form

Dataset Splitting

- `train_test_split()` – to split dataset into training and testing sets

Model Building

- `LinearRegression()` – to create the regression model
- `fit()` – to train the model on training data
- `predict()` – to predict output values

Model Evaluation

- `mean_squared_error()` – to calculate Mean Squared Error
- `mean_absolute_error()` – to calculate Mean Absolute Error
- `r2_score()` – to evaluate model accuracy

Visualization

- `plt.scatter()` – to plot scatter points
- `plt.plot()` – to plot regression line
- `plt.show()` – to display plots

Conclusion:**Post Lab :**

1. Generate synthetic dataset (linear and nonlinear both) for e.g. $y = (x/2) + \sin(x) +$ Gaussian noise or take any function of your choice.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)

# Generate input values
x = np.linspace(0, 10, 200)

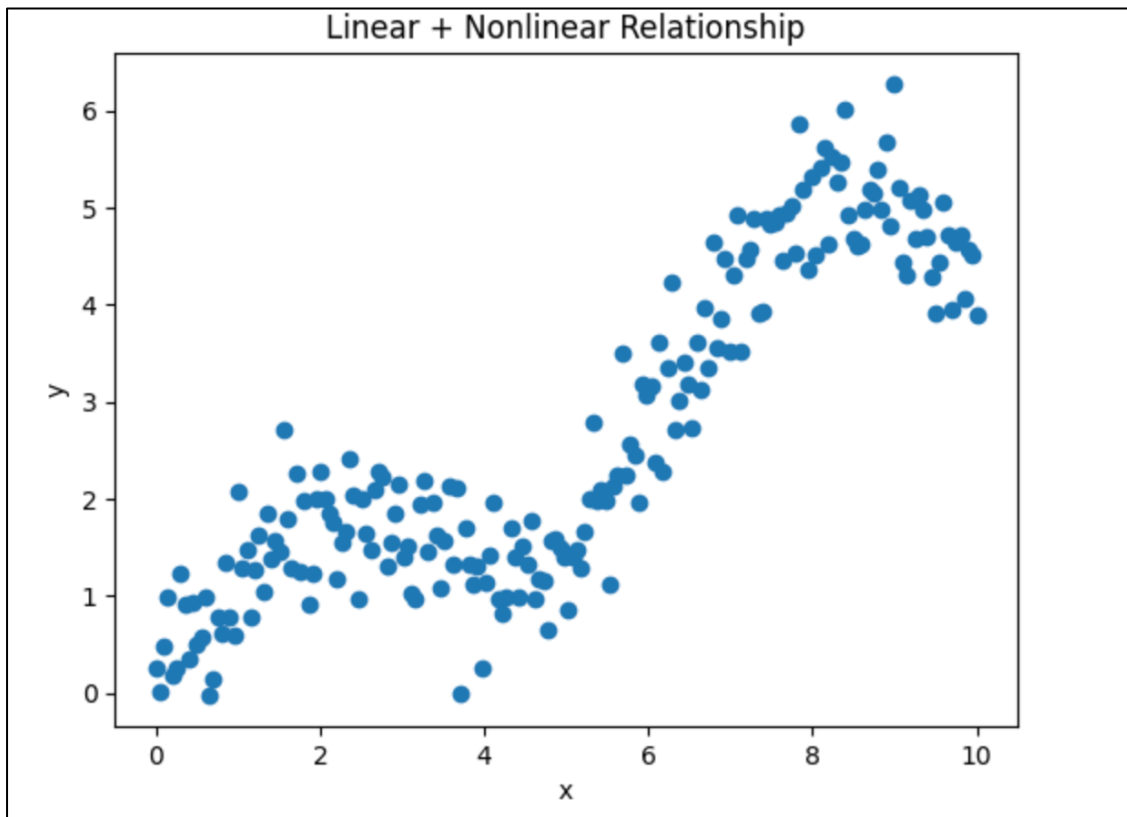
# Gaussian noise
noise = np.random.normal(0, 0.5, size=len(x))

# Output values
y = (x / 2) + np.sin(x) + noise

# Create DataFrame
data = pd.DataFrame({'x': x, 'y': y})
print(data.head())
```

...	x	y
0	0.000000	0.248357
1	0.050251	0.006224
2	0.100503	0.474429
3	0.150754	0.987075
4	0.201005	0.183080

```
plt.scatter(x, y)
plt.title("Linear + Nonlinear Relationship")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,
mean_absolute_error, r2_score
```

```
X = data[['x']]
y = data['y']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?

LinearRegression()



```
slope = regressor.coef_[0]
intercept = regressor.intercept_

print(f"Regression Equation: y = {slope:.3f}x + {intercept:.3f}")
```

Regression Equation: $y = 0.479x + 0.260$

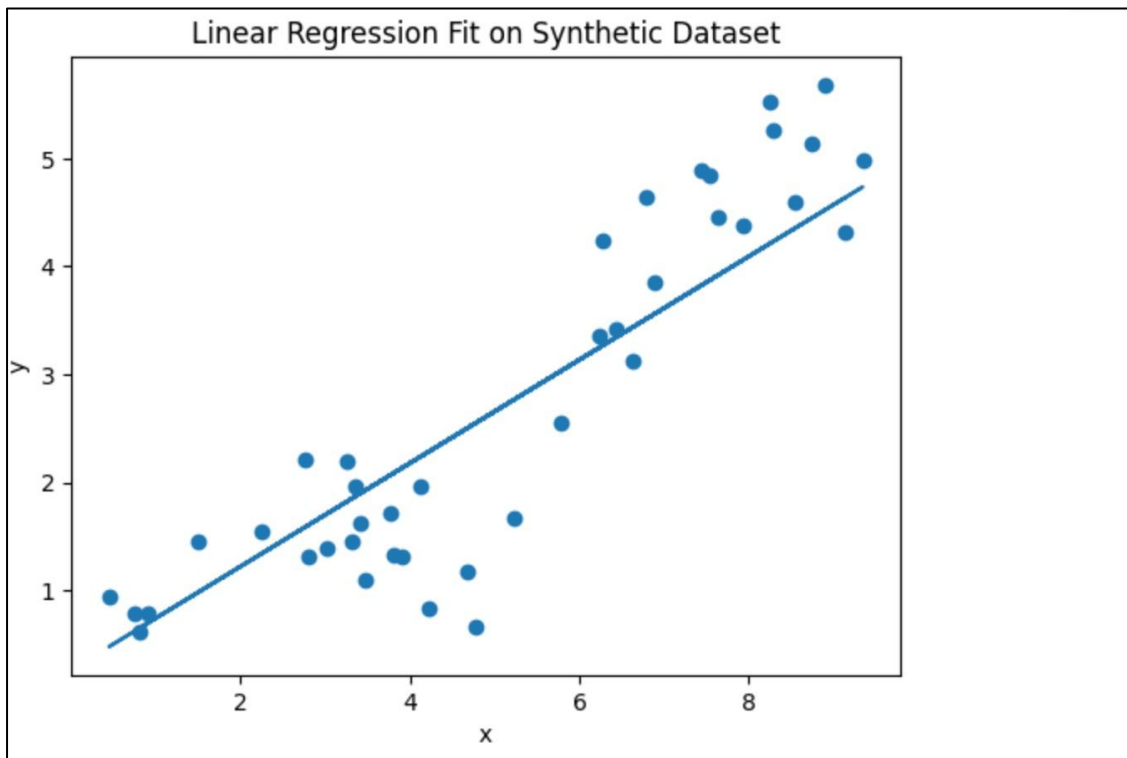
```
y_pred = regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
```

MSE: 0.5568985013857131
RMSE: 0.7462563241847355
MAE: 0.5969353311117697
R2 Score: 0.795353996737871

```
plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred)
plt.title("Linear Regression Fit on Synthetic Dataset")
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```



Comments:

Model Performance Metrics Interpretation

- **Mean Squared Error (MSE):** Measures the average squared difference between actual and predicted values. A non-zero MSE indicates that the model does not perfectly fit the data due to nonlinear behavior and noise.
- **R² Score:** The R² value shows the proportion of variance in y explained by the linear model. Since the dataset contains nonlinear components, the R² score is **moderate**, indicating partial explanation of variance.

Visualization Interpretation

- The regression line captures the **overall linear trend** in the dataset.
- However, it fails to follow the **sinusoidal fluctuations**, leading to visible deviations between predicted and actual values.
- This confirms that linear regression **underfits** the nonlinear data.

2. Fit the regression model using sklearn library. Calculate parameters and metrics compare with linear regression

(Fit the regression model on the data using gradient descent algorithm and its different variants. (Apply linear regression on housing or diabetes dataset or any other dataset and Fit the regression model on the data using gradient descent algorithm .Also compute error values if any.)

Output from Gradient Descent:

```
#Testing the errors
y_test_pred = X_test.dot(w) + b

mse_test = np.mean((y_test - y_test_pred) ** 2)
rmse_test = np.sqrt(mse_test)

print("Test MSE:", mse_test)
print("Test RMSE:", rmse_test)

Test MSE: 2885.6135403538306
Test RMSE: 53.71790707246881
```

Output from Linear Regression:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Train sklearn Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)

LinearRegression()

y_test_pred_sklearn = lr.predict(X_test)

mse_sklearn = mean_squared_error(y_test, y_test_pred_sklearn)
rmse_sklearn = np.sqrt(mse_sklearn)

print("MSE:", mse_sklearn)
print("RMSE:", rmse_sklearn)

... MSE: 2900.1936284934823
    RMSE: 53.853445836765935
```

Comments:

- The close match between error metrics confirms that **Gradient Descent successfully converged** to the optimal solution.
- Both methods minimize the **same Mean Squared Error cost function**, which explains the similarity in results.
- Minor differences arise due to **iterative approximation, learning rate, and numerical precision**.